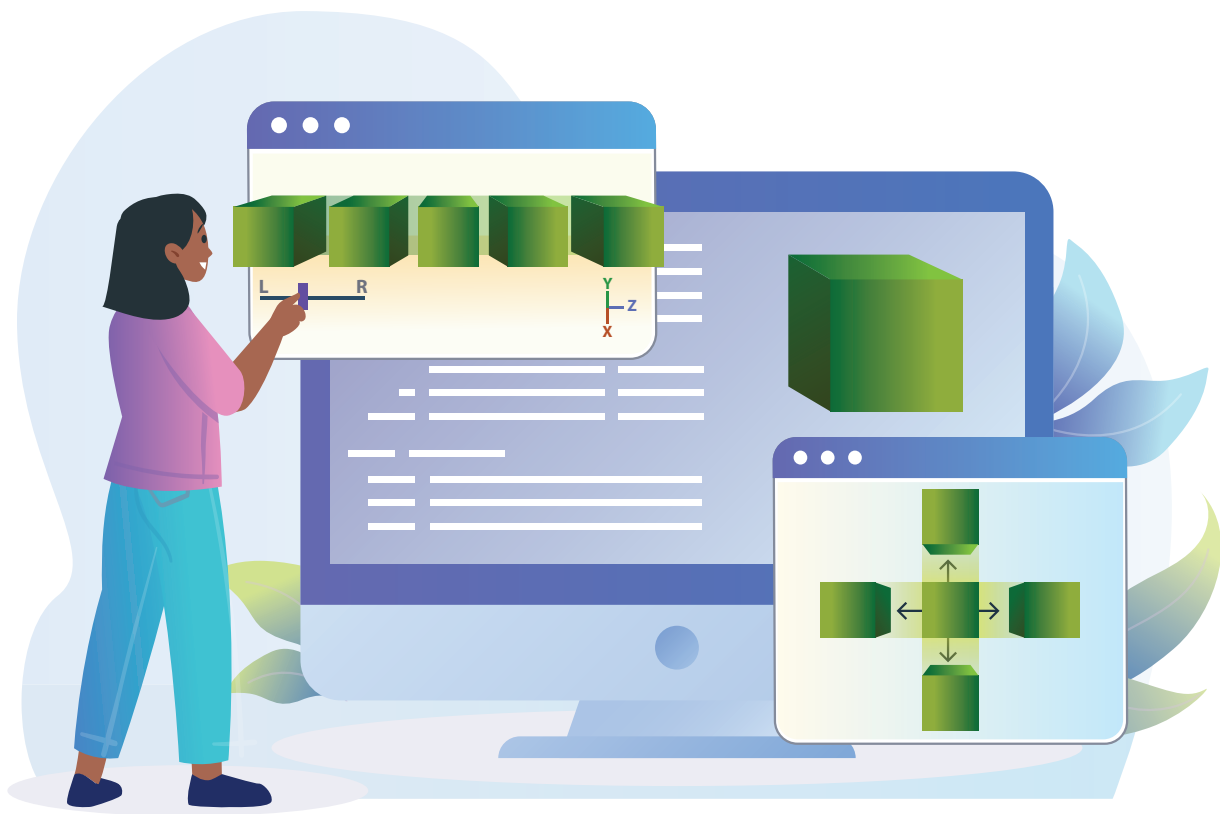


# Transform and Animate Objects



# 02

Transformations and animations are used to create more interactive, visually captivating, and engaging 3D graphics that enhance user experience, simplify complex information, and offer creative flexibility for web applications.



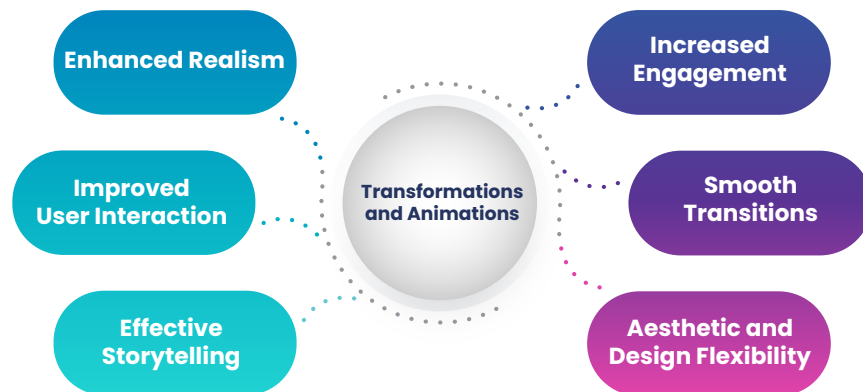
In this lesson, we will delve into the concepts of transformations and animations within Three.js.

Transformations and animations are fundamental in creating engaging and interactive 3D graphics and applications.

## 02. Transform and Animate Objects

### Benefits of Transformations and Animations

- **Enhanced Realism:** By allowing objects to move, rotate, and scale, these techniques mimic real-world physics, making scenes more realistic.
- **Improved User Interaction:** Animations provide visual feedback and interactive elements that make user interfaces more intuitive and responsive.
- **Effective Storytelling:** Transformations and animations help illustrate processes and narratives, making complex concepts easier to understand and follow.



- **Increased Engagement:** Dynamic visual effects and smooth transitions capture users' attention, keeping them engaged and interested in the content.
- **Smooth Transitions:** They ensure that changes between different states or views are fluid and natural, enhancing the overall user experience.
- **Aesthetic and Design Flexibility:** Animations allow for creative and artistic expression, contributing to the unique visual style of applications.

By incorporating transformations and animations, developers and designers can craft more immersive and effective 3D applications that not only look visually appealing but also guide user attention and highlight key elements in the interface.



## Activity

Explore the transformation of an object's position, rotation, and scale, and create simple animations, using native methods as well as with the help of libraries such as 'GSAP and TWEEN.js'.

### Part 1 – Transform Objects

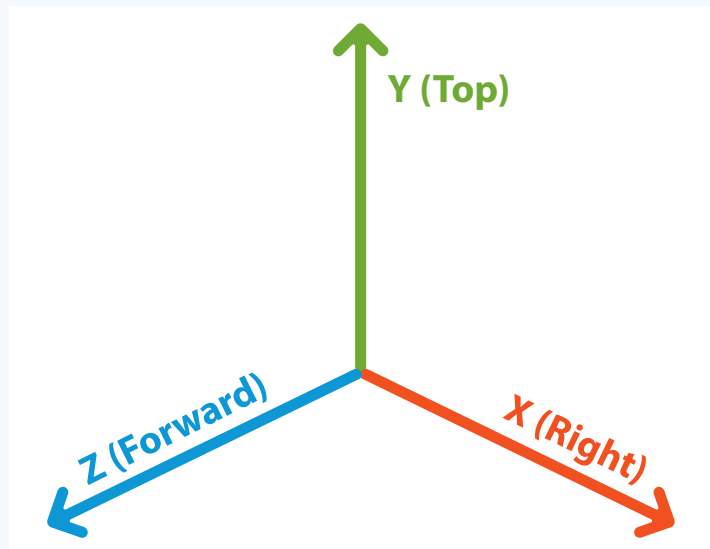
Three main properties can be used to transform objects in the scene.

1. Position
2. Scale
3. Rotation

#### 1. Position objects

The position in 3D space is defined by three key coordinates: **x**, **y**, and **z**. These axes are essential for determining the location of any object in a 3D environment.

In Three.js, the y-axis controls vertical movement (up and down), the z-axis handles depth (forward and backward), and the x-axis manages horizontal positioning (left to right).



Let us continue the code from the previous lesson. If required, download the starter files from the given QR code.



## Activity

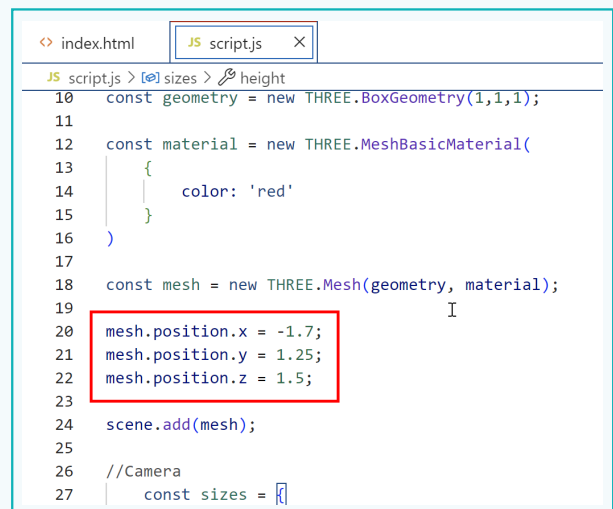
- 1 Open **VSCode** and navigate to the folder created in the previous lesson.

### Note:

If using starter files from the QR code, the following two commands must be executed on the terminal before proceeding ahead. This will create the 'node\_modules' folder.

```
npm install vite
npm install three
```

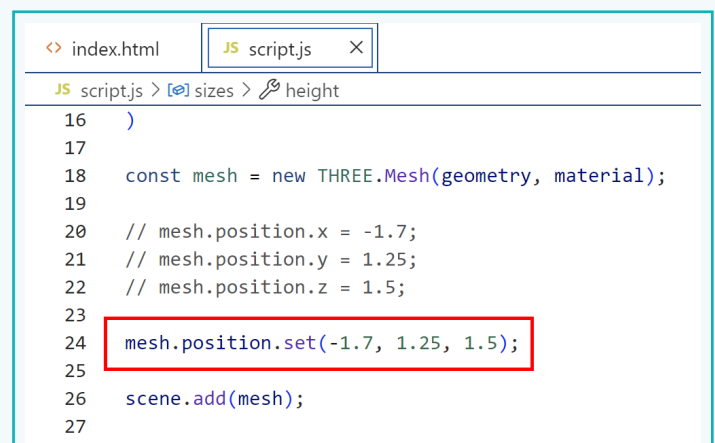
- 2 Go to script.js, and assign different values to the x, y, and z properties of the position class. Check the output.



```
< index.html JS script.js x
JS script.js > [?] sizes > height
10 const geometry = new THREE.BoxGeometry(1,1,1);
11
12 const material = new THREE.MeshBasicMaterial(
13   {
14     color: 'red'
15   }
16 )
17
18 const mesh = new THREE.Mesh(geometry, material);
19
20 mesh.position.x = -1.7;
21 mesh.position.y = 1.25;
22 mesh.position.z = 1.5;
23
24 scene.add(mesh);
25
26 //Camera
27 const sizes = [
```

Ensure that these statements are written before the call to the render function.

- 3 Instead of adjusting x, y, and z individually, use the `.set()` method to assign the values to the 3 axes at once.



```
< index.html JS script.js x
JS script.js > [?] sizes > height
16 )
17
18 const mesh = new THREE.Mesh(geometry, material);
19
20 // mesh.position.x = -1.7;
21 // mesh.position.y = 1.25;
22 // mesh.position.z = 1.5;
23
24 mesh.position.set(-1.7, 1.25, 1.5);
25
26 scene.add(mesh);
27
```

## Activity

### Useful Methods in the Position Class:

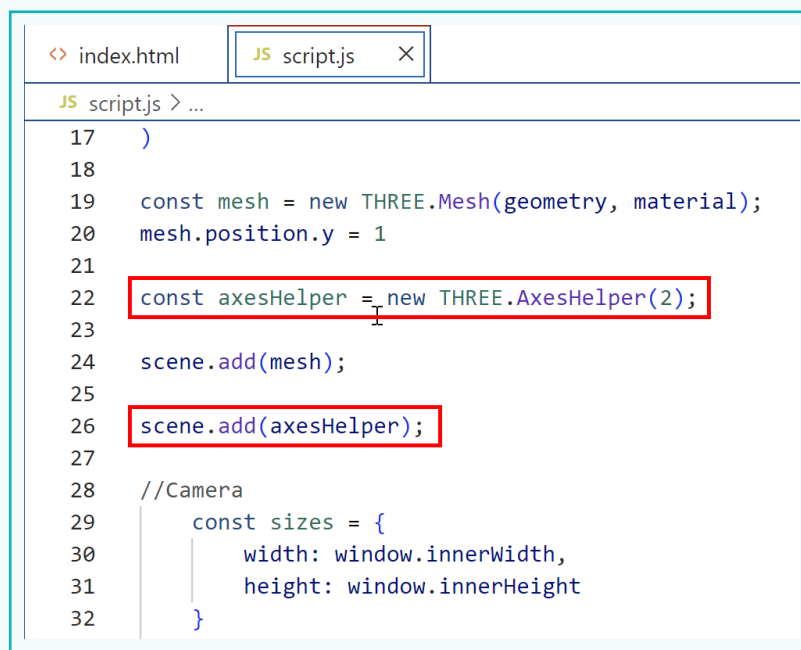
- `mesh.position.length()` - Determines the length of a vector.
- `mesh.position.distanceTo(camera.position)` - Calculates the distance from another Vector3 (e.g., camera).
- `mesh.position.normalize` - Normalizes the vector (reducing its length to 1 unit while maintaining its direction).

### Axes Helper

Positioning objects in 3D space can be tricky. Especially when moving the camera, keeping track of axis orientation becomes more challenging. A helpful solution in Three.js is the *AxesHelper*.

The *AxesHelper* visually represents the x, y, and z axes with three lines, each originating from the center of the scene and extending in their respective directions.

To use it, create an instance of the *AxesHelper* class and add it to the scene immediately after the scene is instantiated. The line length can be set by passing it as a parameter. In this case, a length of 2 will be used.

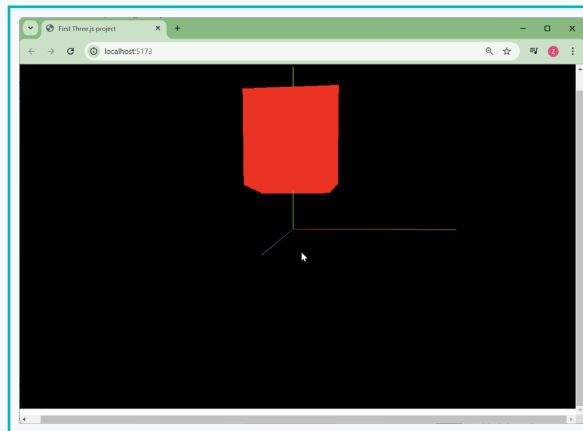


```
<> index.html JS script.js X
JS script.js > ...
17 )
18
19 const mesh = new THREE.Mesh(geometry, material);
20 mesh.position.y = 1
21
22 const axesHelper = new THREE.AxesHelper(2);
23
24 scene.add(mesh);
25
26 scene.add(axesHelper);
27
28 //Camera
29 const sizes = {
30   width: window.innerWidth,
31   height: window.innerHeight
32 }
```

## Activity

Initially, only green and red lines will be visible.

The green line represents the y-axis, and the red line represents the x-axis. There is also a blue line for the z-axis, but it is not visible because it is aligned with the camera. It will become visible once the scene is rotated, or the camera is moved.



The Axes Helper can be used as a visual guide for reference.

## 2. Scale Objects

In Three.js, **scaling** allows resizing of objects along the x, y, and z axes. By default, an object's scale is set to 1 on all axes, meaning it appears at its original size. These values can be modified to increase or shrink the object in specific directions.

- 4 Adjust the individual axes by using the scale property. Try different values for each axis.

```
index.html JS script.js X
JS script.js > sizes > height

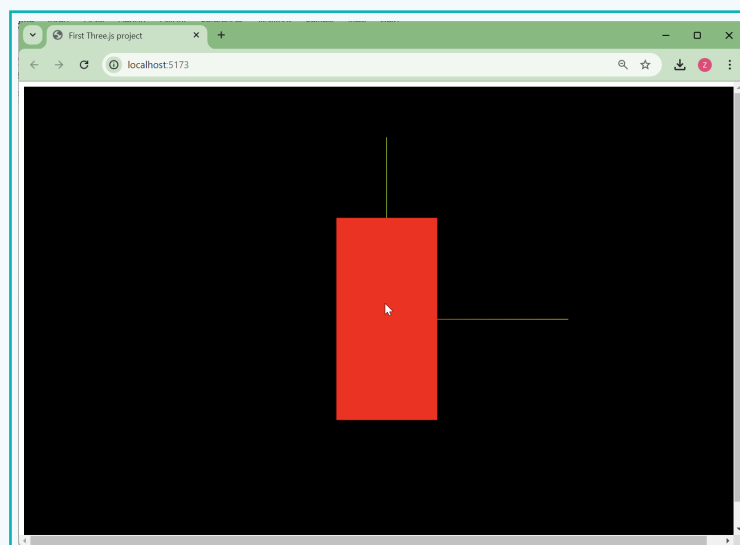
20 const axesHelper = new THREE.AxesHelper(3);
21
22 // mesh.position.x = -1.7;
23 // mesh.position.y = 1.25;
24 // mesh.position.z = 1.5;
25
26 // mesh.position.set(-1.7, 1.25, 1.5);
27
28 mesh.scale.x = 1;
29 mesh.scale.y = 2;
30 mesh.scale.z = 4;
31
32 scene.add(mesh);
33
34 //Camera
35 const sizes = {
36   width: window.innerWidth,
37   height: window.innerHeight,
38 }
```

### Activity

- 5 Instead of setting values for each axis individually, use the `.set()` method to assign values to all three axes at once.

```
<> index.html  JS script.js  X
JS script.js > ...
25
26 // mesh.position.set(-1.7, 1.25, 1.5);
27
28 // mesh.scale.x = 1;
29 // mesh.scale.y = 2;
30 // mesh.scale.z = 4;
31
32 mesh.scale.set(1,2,4);
33
34 scene.add(mesh);
35
36 //Camera
37   const sizes = {
38     width: window.innerWidth,
39     height: window.innerHeight
40   }
```

Output -

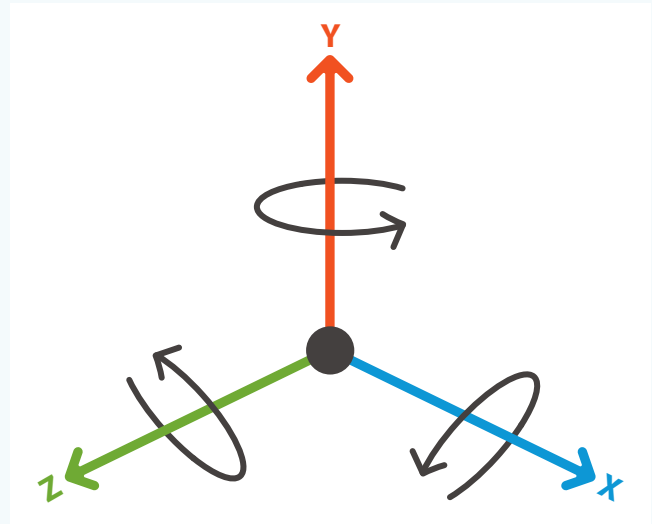


## Activity

### 3. Rotate Objects

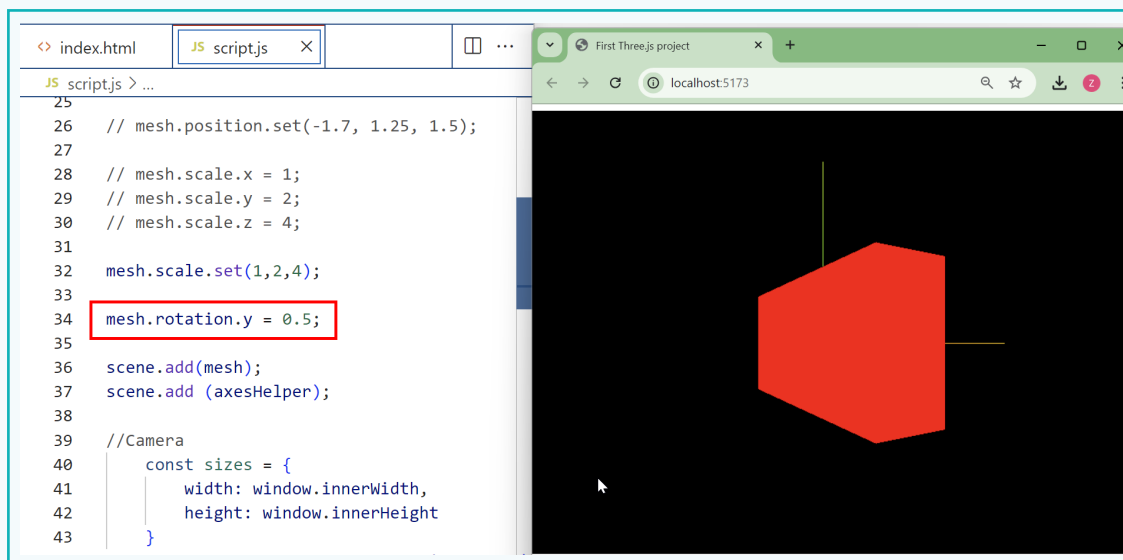
The *rotation* property allows manipulation of an object's orientation in 3D space. This property stores the rotation of an object in radians along each of the three axes.

- X-axis rotation: This is like nodding your head up and down. The object rotates forward and backward.
- Y-axis rotation: This is like shaking your head left and right. The object rotates horizontally.
- Z-axis rotation: This is like the movement of a clock's hands, rotating clockwise or counterclockwise.



Let us try to rotate the cube by half a turn around the y-axis.

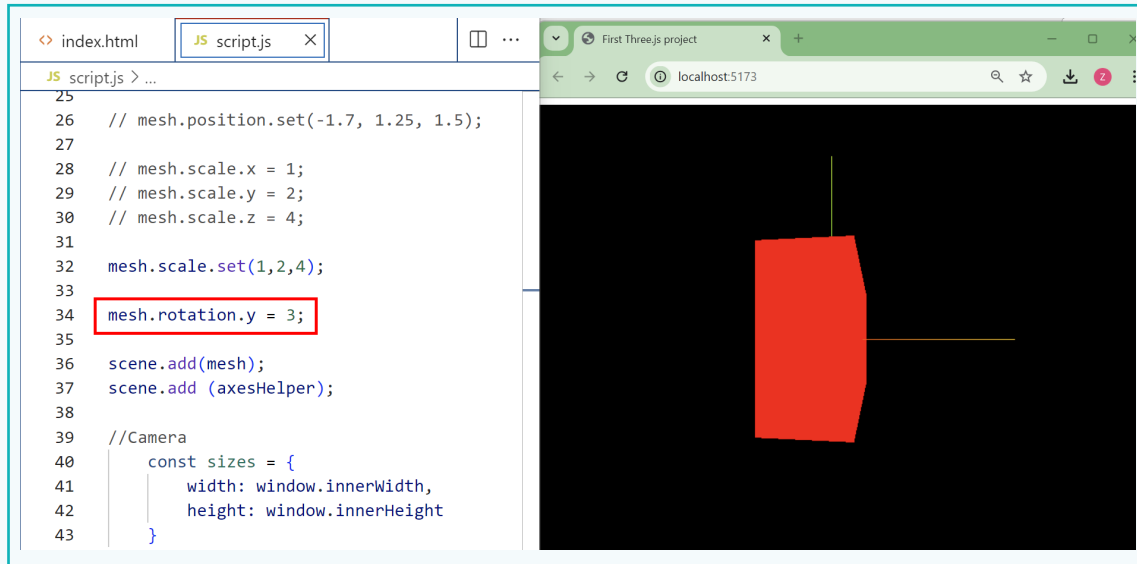
- 6 Rotate the cube around the y-axis using the rotation property as shown. Start with a value of 0.5 radians.





### Activity

- 7 Gradually increase the value until the cube completes a semi-circle.



It will be observed that a half-circle rotation occurs at approximately **3.14** radians, which is the value of  **$\pi$  (pi)**.

$\pi$  is a key value for rotations:

- A half rotation ( $180^\circ$ ) =  **$\pi$  radians**
- A full rotation ( $360^\circ$ ) =  **$2\pi$  radians.**

In JavaScript, this can be utilized using the *Math.PI* property.

### Try

Figure out the value needed for a quarter rotation.

**Answer:**  $\text{Math.PI}/2$  or  $\text{Math.PI} * 0.5$

Let us apply these rotation concepts to the cube in the scene.

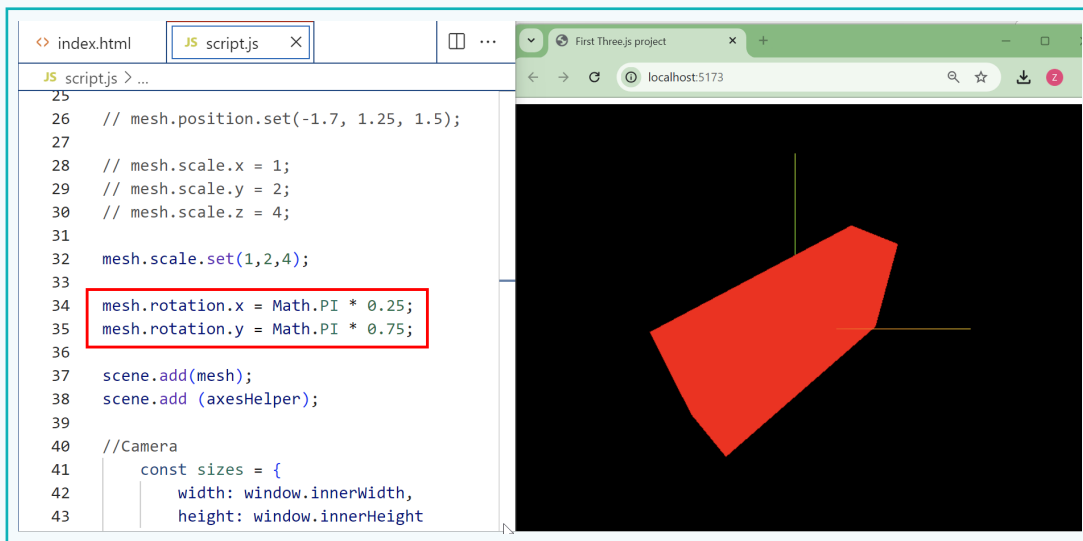
- 8 Add a  $45^\circ$  rotation on the x-axis or a one-eighth of a complete rotation.

This is equivalent to half of a quarter rotation.

Hence, One eighth =  **$\text{Math.PI}/4$**  or  **$\text{Math.PI} * 0.25$ .**

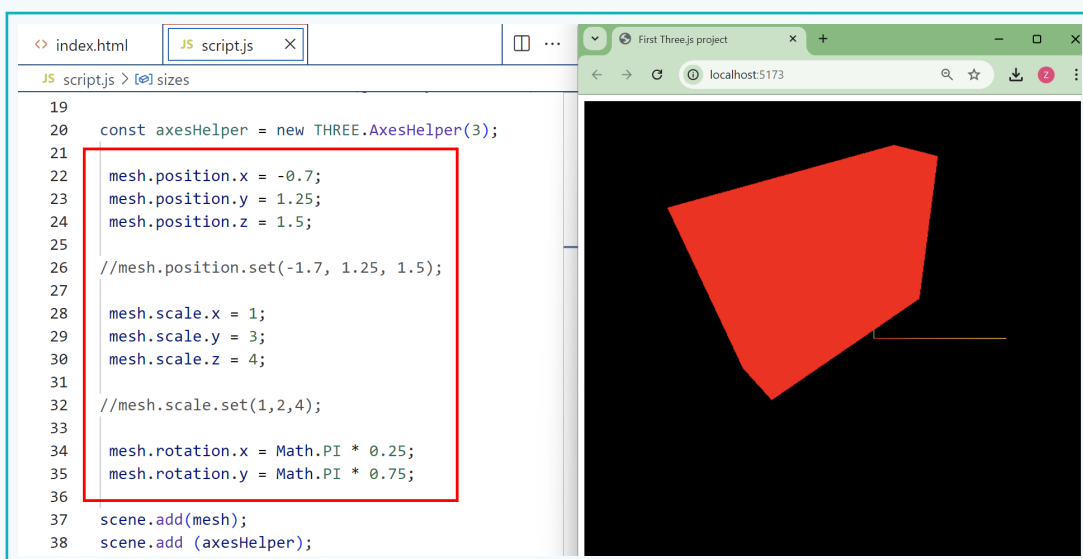
### Activity

- 9 Also, add a 135° rotation on the y-axis.  
Three-fourth =  $\text{Math.PI} * 3/4$  or  $\text{Math.PI} * 0.75$ .



Feel free to experiment with different rotation values.

- 10 Combine all the transformations that have been done so far.



## Activity

### Note:

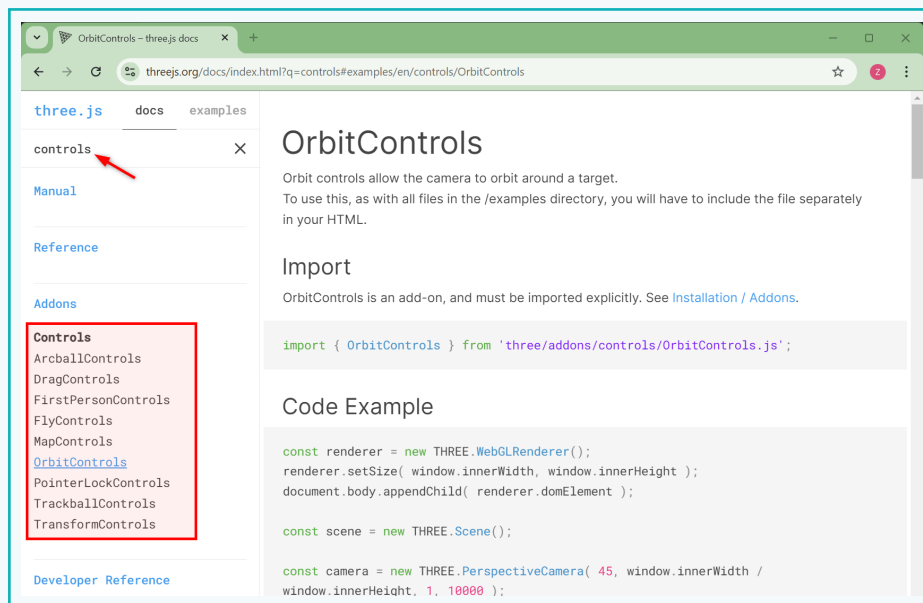
The position, rotation, and scale transformations can be combined in any order. Regardless of the order in which these transformations are combined, the final result will remain the same.

## Part 2 - Animation

In the previous lesson, we used 'controls' that allowed us to navigate the scene like in a 3D editor.

Let us understand this in more detail.

Three.js offers various classes for controlling the camera and more. These can be explored by typing 'controls' in the three.js documentation.



We have imported the *OrbitControls* class in the previous lesson. Let us understand more about it.

```
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
```

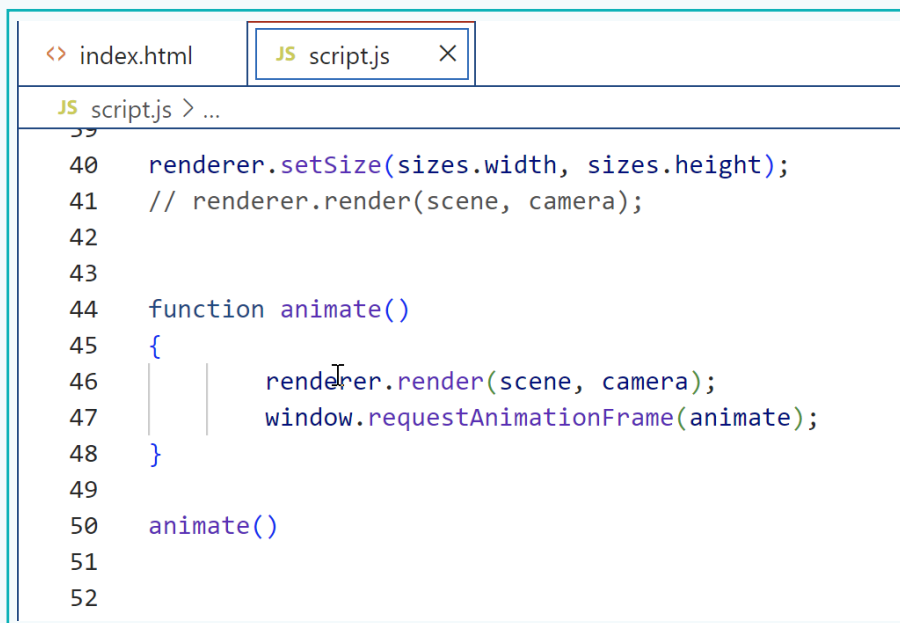
This class provides a set of methods that enable users to orbit, pan, and zoom around 3D objects in the scene using a computer mouse or trackpad.

### Activity

To use the orbit control system in the code, an instance of the *OrbitControls* class is created and assigned to a constant variable.

```
const controls = new OrbitControls(camera, canvas);
```

Next, we defined a function `animate()`, which includes the ***requestAnimationFrame()*** method.



```
<> index.html  JS script.js  X
JS script.js > ...
39
40  renderer.setSize(sizes.width, sizes.height);
41  // renderer.render(scene, camera);
42
43
44  function animate()
45  {
46      |       renderer.render(scene, camera);
47      |       window.requestAnimationFrame(animate);
48  }
49
50  animate()
51
52
```

Let us explore *requestAnimationFrame()* in more detail.

When using *three.js* controls, users can interact with a 3D scene (rotate, zoom, pan, etc.). These interactions must be updated every frame to create a seamless experience. Continuous updates are necessary to reflect user input changes and maintain smooth, real-time interactions with the scene.

Controls like *OrbitControls* have a *.update()* method that needs to be called on every frame to update the camera position based on user input. This method manages camera movements, inertia, damping, and other effects.

This can be achieved using the native JavaScript function *requestAnimationFrame()*. This method allows these updates to happen seamlessly.

### Activity

#### Note:

A **native** function is a function that is built into the JavaScript language and provided by the JavaScript runtime environment (such as a web browser).

`requestAnimationFrame()` is used to create an animation loop that continuously updates the scene, checking for any changes in user input.

By using `requestAnimationFrame()` with controls, we ensure:

#### Responsive Feedback

Every user action (like dragging to rotate the scene) is immediately reflected in the viewport, providing instant feedback.

#### Consistent Behaviour

Regardless of the device's performance, the animation loop runs at an optimal rate, maintaining a consistent experience across different hardware.

#### Impact of not using `requestAnimationFrame()` in Three.js

- **No Continuous Updates:** Without `requestAnimationFrame()`, the control's `.update()` method is not called regularly. Consequently, the scene does not continuously reflect user inputs, causing the controls to appear unresponsive or malfunctioning.
- **Static Scene:** If `requestAnimationFrame()` is removed and replaced with a single call to the `.update()` and `renderer.render(scene, camera)`, the scene will only render once. It will not update dynamically in response to user actions unless manual re-renders are forced, which is inefficient and impractical.

We can use the `requestAnimationFrame()` method to create simple animations by placing the transform statements for position, rotation, and scale, inside the `animate()` function.

We will use the `'+='` or `'-='` operators to continually increment or decrement the transformation values within the `animate()` loop, creating a smooth, ongoing animation effect.

### Activity

To create simple animations using the `requestAnimationFrame()` method, follow these steps:

- 1 Place the transform statements for position, rotation, and scale inside the `animate()` function.
- 2 Use the `'+='` or `'-='` operators to continuously increment or decrement the transformation values within the `animate()` loop.

This will generate a smooth, ongoing animation effect.

```
63
64  function animate()
65  {
66      mesh.scale.y += 0.001;
67      renderer.render(scene, camera);
68      window.requestAnimationFrame(animate);
69  }
70
71  animate()
--
```

Output -



### Bytes

The **lookAt()** method orients an object to face a specific target, making its -z axis point towards the target's coordinates. This is useful for tracking moving objects, rotating the camera, orienting the player to always face an enemy, or moving the character's eyes to an object.

#### Syntax:

`object.lookAt(x, y, z)` OR `object.lookAt(vector)`

Here,

- x, y, z are target coordinates.
- vector is a `THREE.Vector3` object specifying the target.

### Activity

#### Part 3 - Animations using TWEEN

JavaScript provides a simple and lightweight animation library – **TWEEN.js**, which allows smooth transitions (tweening) between numerical values over time. It is particularly useful for animating properties such as positions, rotations, scales, and other values in both 2D and 3D contexts. **TWEEN.js** can be used to animate anything, from DOM elements to WebGL/Three.js objects, or even arbitrary JavaScript objects.

A **tween** (short for ‘in-between’) in animation refers to generating intermediate frames between two keyframes, creating a smooth transition. This technique is used to animate objects by moving them between positions, rotating, scaling, or changing other properties.

Following steps are used when the Tween library is used to create an animation –

#### Create a Tween

Specify the object (or value) to animate and the target value for transition.

#### Define Duration

Specify how long the transition should take.

#### Start the Animation

The tween runs in an update loop, calculating intermediate values at each frame to smoothly transition from the start to the end value.

## Activity

Let us incorporate it into our code.

Include the library in the code either by installing it through npm, similar to Vite and Three.js libraries or by adding it to the HTML file via a CDN (Content Delivery Network), which provides fast and easy access to the library without needing to download or manage the files yourself.

- 1 Include Tween.js via a CDN link in the HTML file.

```
<> index.html X JS script.js
<> index.html > ...
3 <html lang="en">
4 <head>
7 <title> First Three.js project </title>
8 </head>
9 <body>
10 <!-- <h1> Three.js website - Under Construction </h1> -->
11
12 <canvas class="webgl"></canvas>
13
14 <script type="module" src="./script.js"></script>
15 <script src="https://cdnjs.cloudflare.com/ajax/libs/tween.js/18.6.4/tween.umd.js"></script>
16 </body>
17 </html>
18
19
```

The cdn link is as below –

<https://cdnjs.cloudflare.com/ajax/libs/tween.js/18.6.4/tween.umd.js>

Next, create a target position for the mesh.

- 2 Define an object variable with three properties: x, y, and z, each representing a coordinate in 3D space. Assign the appropriate values. The mesh will get animated between its initial position and this target position.

```
<> index.html JS script.js X
JS script.js > ...
12 const material = new THREE.MeshBasicMaterial(
16 )
17
18 const mesh = new THREE.Mesh(geometry, material);
19 const axesHelper = new THREE.AxesHelper(3);
20
21 const meshTargetPosition = { x: 2, y: 0, z: 5 };
22
23 // mesh.position.x = -0.7;
24 // mesh.position.y = 1.25;
25 // mesh.position.z = 1.5;
26
27 mesh.position.set(-1.7, 1.25, 1.5);
28
```



### Activity

- 3 Create a new 'tween' object that will animate the mesh position. Use the `.to()` method to move the mesh from its current coordinates to the target coordinates. Finally, use the `start()` method to start the tween animation immediately.

```
<> index.html JS script.js X
JS script.js > ...
57 const renderer = new THREE.WebGLRenderer({
58   canvas: canvas,
59 })
60
61 renderer.setSize(sizes.width, sizes.height);
62
63 // renderer.render(scene, camera);
64
65 new TWEEN.Tween(mesh.position)
66   .to(meshTargetPosition, 2000) // Move to target position over 2 seconds
67   .start(); // Start the tween
68
69 function animate(time)
70 {
71   // mesh.scale.y += 0.001;
72   // mesh.rotation.x += 0.01;
73   // mesh.rotation.y += 0.01;
```

- 4 Call the `TWEEN.update(time)` method to update the values of a tween over time in the `animate()` method loop.

In each iteration of this loop, the `TWEEN.update(time)` calculates how much time has passed since the last update and determines the current state of each tween. It then applies the necessary changes to the animated properties based on the elapsed time.

```
function animate(time)
{
  // mesh.scale.y += 0.001;
  // mesh.rotation.x += 0.01;
  // mesh.rotation.y += 0.01;
  renderer.render(scene, camera);

  TWEEN.update(time);

  window.requestAnimationFrame(animate);
}

animate()
```

### Activity

Output -



Although Tween.js is a minimalistic and efficient tool for creating smooth animations by interpolating between values, it lacks advanced features like timelines, complex sequencing, or direct DOM manipulation that some other comprehensive libraries offer.

Let us explore one such comprehensive library – **GSAP**.

#### Part 4 - Animations using GSAP

At times, you may want to animate your scene in a way that requires an additional library. Among the many animation libraries available, GSAP is one of the most popular options.

GSAP (GreenSock Animation Platform) is a powerful and highly flexible JavaScript library designed for creating fast, smooth animations on the web. It is widely used in web development, especially for animating HTML elements, CSS properties, SVGs, and even 3D objects in frameworks like Three.js.

To use GSAP in the project, it can either be added through npm, similar to Vite and Three.js, or included via a CDN, like Tween.js.

To install it using npm, run the following command on the terminal.

```
npm install gsap
```

Once this command is executed successfully, GSAP will be available in the 'node\_modules' folder. It can then be imported into the script using the import statement.

```
import gsap from 'gsap'
```

## Activity

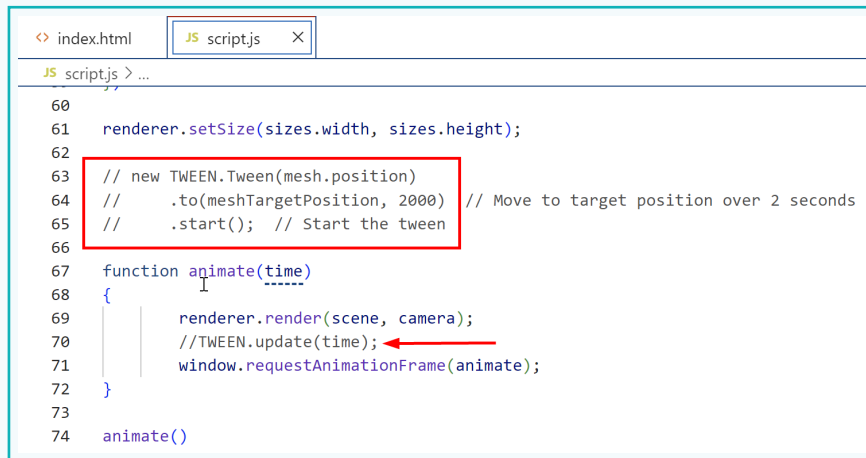
- 1 Include the library via CDN by adding GSAP in the <head> or <body> section of the HTML file.



```
<? index.html X JS script.js
<? index.html > ...
3 <html lang="en">
8 </head>
9 <body>
10 <!-- <h1> Three.js website - Under Construction </h1> -->
11
12 <canvas class="webgl"></canvas>
13
14 <script type="module" src="./script.js"></script>
15 <script src="https://cdnjs.cloudflare.com/ajax/libs/tween.js/18.6.4/tween.umd.js"></script>
16
17 <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.11.3/gsap.min.js"></script>
18
19 </body>
20 </html>
```

The link is as given below - <https://cdnjs.cloudflare.com/ajax/libs/gsap/3.11.3/gsap.min.js>

- 2 Next, go to 'script.js' and comment out the lines of code related to Tween.js that were added in the previous step.



```
<? index.html JS script.js X
JS script.js > ...
60
61 renderer.setSize(sizes.width, sizes.height);
62
63 // new TWEEN.Tween(mesh.position)
64 // .to(meshTargetPosition, 2000) // Move to target position over 2 seconds
65 // .start(); // Start the tween
66
67 function animate(time)
68 {
69     renderer.render(scene, camera);
70     //TWEEN.update(time);
71     window.requestAnimationFrame(animate);
72 }
73
74 animate()
```

- 3 Call the *gsap.to()* method. This method is used to animate a property from its current value to a specified value.

## Activity

The syntax of this method is as below –

`gsap.to(target, vars)`

Here,

- **target** specifies the object or property to animate.
- **vars** is an object that specifies the properties to animate and their values, as well as other animation settings.

```
<> index.html JS script.js X
JS script.js > ...
66 // .to(meshTargetPosition, 2000) // Move to t:
67 // .start(); // Start the tween
68
69
70 gsap.to(mesh.position, {
71   x: 2,
72   duration: 2,
73 });
74
75 function animate(time) I
76 {
77   renderer.render(scene, camera);
78   //TWEEN.update(time);
79   window.requestAnimationFrame(animate);
80 }
81
```

In the above code,

**x** refers to the target value for the 'x' property of mesh.position. GSAP will animate the 'x' coordinate of the mesh's position from its current value to 2.

**duration** specifies how long the animation should take, in seconds. Here, the animation will last for 2 seconds.

Output -



### Activity

Let us make the animation a bit more interesting by looping the animation in a yoyo effect.

- Add the below marked lines of code.

```
gsap.to(mesh.position, {  
  x: 2,  
  duration: 2,  
  yoyo: true, ←  
  repeat: -1, ←  
});
```

Here,

*yoyo: true* makes the animation reverse back to the starting value when it reaches the end.

*repeat: -1* specifies how many times the animation should repeat. Setting repeat to -1 means the animation will repeat indefinitely. With *yoyo: true*, this creates a continuous back-and-forth animation.

 Output -



Try experimenting with additional properties to create a variety of animation effects.

## 02. Transform and Animate Objects



### Choosing the Right Library

Choosing the right animation library involves understanding the project's requirements and evaluating available options based on functionality, performance, ease of use, support, and testing to ensure it meets your needs. By carefully considering these factors, you can choose the library that best fits the project and development goals.

Here is a simple comparison between the two libraries explored in this lesson:

| Feature     | Tween.js                                | GSAP                                           |
|-------------|-----------------------------------------|------------------------------------------------|
| Size        | Lightweight (~11KB)                     | Larger (~50KB), more feature-rich              |
| Ease of Use | Simple API, Manual loop handling        | Automatic loop, more intuitive API             |
| Features    | Basic tweening, custom properties       | Advanced features, CSS/DOM animations, SVG, 3D |
| Timeline    | No timeline                             | Powerful timeline and sequencing               |
| Plugins     | None                                    | Extensive plugin ecosystem                     |
| Performance | Great for lightweight custom animations | Optimized for high performance                 |
| Use Case    | Simple numeric and canvas animations    | Complex web animations, DOM, UI                |

Keep experimenting with different properties and techniques to discover new ways to animate the scenes effectively.

In this lesson, we explored how to bring Three.js scenes to life with transitions and animations. We understood fundamental techniques such as using `requestAnimationFrame()` for manual updates and how libraries like TWEEN.js and GSAP can simplify and enhance animations.

## Exercise

 State if the statements are True or False.

- 1 The `set()` method can be used to assign values in one go.  
 \_\_\_\_\_
- 2 If you rotate on the x-axis, the object will rotate in the up-down direction.  
 \_\_\_\_\_
- 3 When we combine position, rotation, and scale transformations, the result will differ depending on the order.  
 \_\_\_\_\_
- 4 `requestAnimationFrame()` is used to create an animation loop that continuously updates the scene, checking for any changes in user input.  
 \_\_\_\_\_

 Tick mark the correct answer.

- 5 When using the Axes Helper, the green line represents the \_\_\_\_\_.  
a. x-axis      b. y-axis      c. z-axis      d. w-axis
- 6 The value needed for a full rotation is \_\_\_\_\_.  
a. `Math.PI`      b. `2*Math.PI`      c. `Math.PI/2`      d. `Math.PI/4`
- 7 \_\_\_\_\_ provides a set of methods that enable users to orbit, pan, and zoom around 3D objects in the scene using a computer mouse or trackpad.  
a. `OrbitControls`      b. `TransformControls`      c. `FlyControls`      d. `LookAt`

## Exercise

8 A \_\_\_\_\_ refers to the process of generating intermediate frames between two keyframes.

- a. gsap      b. render      c. tween      d. import

Answer the following questions in one line.

9 How is the axes helper useful?

---

---

10 State the steps typically followed for creating an animation using the tween library.

---

---

## Tech Flash

|                                |                                                                                                            |
|--------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Axes Helper</b>             | : It is a visual reference for representing the x, y and z axes.                                           |
| <b>OrbitControls</b>           | : It provides a set of methods that enable users to orbit, pan, and zoom around 3D objects in the scene.   |
| <b>requestAnimationFrame()</b> | : It creates an animation loop that continuously updates the scene.                                        |
| <b>tween.js</b>                | : It is a simple and lightweight animation library to create smooth transitions.                           |
| <b>GSAP</b>                    | : It is a powerful and highly flexible JavaScript library for creating fast, smooth animations on the web. |